

Pràctica 9: Inicialització de paràmetres

Laura Rodríguez Cima – Víctor Navas Portella

Grau d'Estadística Aplicada

Curs 2025/2026

Objectiu

En aquesta pràctica estudiarem com la **inicialització dels pesos** afecta l'entrenament d'una xarxa neuronal.

Què farem avui?

- Entendre per què no és bona idea inicialitzar els pesos de qualsevol manera.
- Interpretar els pesos inicials com a **variables aleatòries**.
- Entendre l'objectiu de mantenir estable la **variància** del senyal.
- Implementar manualment inicialitzacions **Xavier/Glorot** i **He/Kaiming**.
- Comparar inicialitzacions petites, grans, Xavier i He.
- Visualitzar com canvien les **activacions** capa a capa.
- Aplicar aquestes inicialitzacions al problema `two_moons`.

La idea fonamental és que una bona inicialització ajuda a mantenir estable el senyal en el **forward pass** i també els gradients en el **backward pass**.

Per què importa la inicialització?

Quan entrenem una xarxa neuronal, els pesos no comencen amb valors apresos. Cal donar-los un valor inicial.

Si els pesos són massa petits:

- les activacions poden anar-se fent molt petites,
- els gradients poden desaparèixer,
- l'entrenament pot ser molt lent.

Si els pesos són massa grans:

- les activacions poden explotar,
- la loss pot ser inestable,
- l'optimitzador pot tenir dificultats per convergir.

Idea clau

Inicialitzar bé vol dir començar amb pesos aleatoris, però amb una escala coherent amb les dimensions de cada capa.

Els pesos com a variables aleatòries

Abans d'entrenar, no sabem quins pesos seran bons. Per això, inicialitzem cada pes com una **variable aleatòria**.

Per exemple, per a una capa l :

$$\theta_{ij}^{(l)} \sim \mathcal{N}(0, \sigma^2)$$

Això vol dir que:

- els pesos es generen aleatòriament,
- estan centrats a zero,
- la seva dispersió depèn de σ^2 ,
- controlar σ^2 equival a controlar l'escala inicial dels pesos.

Per què no posar tots els pesos a zero?

Si totes les neurones comencen igual, faran el mateix càlcul i rebran gradients molt semblants. La xarxa no trenca la simetria.

Distribucions habituals

Normalment utilitzem distribucions centrades a zero.

Distribució normal

$$\theta_{ij}^{(l)} \sim \mathcal{N}(0, \sigma^2)$$

El paràmetre important és σ^2 , la **variància**.

Distribució uniforme

$$\theta_{ij}^{(l)} \sim U(-a, a)$$

En aquest cas:

$$\text{Var}(\theta) = \frac{a^2}{3}$$

Per tant, per obtenir una variància concreta, triem a adequadament.

En aquesta pràctica implementarem les dues opcions, però començarem sobretot amb la versió normal.

Objectiu teòric: mantenir la variància

Considerem una pre-activació qualsevol:

$$z_i^{(l)} = \sum_{j=1}^{D^{(l-1)}} \theta_{ij}^{(l)} h_j^{(l-1)}$$

La mida de $z_i^{(l)}$ depèn de:

- el nombre d'entrades de la capa anterior, $D^{(l-1)}$,
- l'escala dels pesos $\theta_{ij}^{(l)}$,
- l'escala de les activacions $h_j^{(l-1)}$.

Condicció desitjada

Volem que la variància del senyal no canviï massa en passar d'una capa a la següent:

$$\text{Var}(h^{(l)}) \approx \text{Var}(h^{(l-1)})$$

Això ajuda a evitar tant el **vanishing** com l'**exploding** del senyal i dels gradients.

Variància al forward pass

Amb les assumpcions habituals:

- pesos independents,
- pesos centrats: $\mathbb{E}[\theta_{ij}^{(l)}] = 0$,
- activacions independents dels pesos,

obtenim aproximadament:

$$\text{Var}(z_i^{(l)}) = D^{(l-1)} \text{Var}(\theta^{(l)}) \text{Var}(h^{(l-1)})$$

Conseqüència

Per mantenir estable el forward pass, voldríem:

$$\text{Var}(\theta^{(l)}) \approx \frac{1}{D^{(l-1)}}$$

Aquesta és la idea del **fan-in**: si una capa rep moltes entrades, cada pes individual ha de ser més petit.

Variància al backward pass

Durant el backpropagation, l'error es propaga cap enrere multiplicant per matrius de pesos transposades.

De manera anàloga, per l'error local podem obtenir:

$$\text{Var}(\delta^{(l-1)}) = D^{(l)} \text{Var}(\theta^{(l)}) \text{Var}(\delta^{(l)})$$

Conseqüència

Per mantenir estable el backward pass, voldríem:

$$\text{Var}(\theta^{(l)}) \approx \frac{1}{D^{(l)}}$$

Problema: si $D^{(l-1)} \neq D^{(l)}$, no podem satisfer exactament les dues condicions alhora.

Inicialització Xavier / Glorot

La inicialització **Xavier/Glorot** proposa una solució de compromís entre l'estabilitat del forward i la del backward.

Si una capa té:

- $D^{(l-1)}$ entrades,
- $D^{(l)}$ sortides,

la variància recomanada és:

$$\text{Var}(\theta^{(l)}) = \frac{2}{D^{(l-1)} + D^{(l)}}$$

Versió normal

$$\theta_{ij}^{(l)} \sim \mathcal{N}\left(0, \sigma^2 = \frac{2}{D^{(l-1)} + D^{(l)}}\right)$$

Versió uniforme

$$\theta_{ij}^{(l)} \sim U\left(-\sqrt{\frac{6}{D^{(l-1)} + D^{(l)}}}, \sqrt{\frac{6}{D^{(l-1)} + D^{(l)}}}\right)$$

Inicialització He / Kaiming

Amb activació **ReLU**, aproximadament la meitat dels valors negatius queden anul·lats:

$$\text{ReLU}(z) = \max(0, z)$$

Per això, es pot perdre aproximadament una part important de la variància capa a capa.
En xarxes profundes amb ReLU, Xavier pot quedar-se curta.

La inicialització **He/Kaiming** compensa aquest efecte utilitzant:

$$\text{Var}(\theta^{(l)}) = \frac{2}{D^{(l-1)}}$$

Versió normal

$$\theta_{ij}^{(l)} \sim \mathcal{N}\left(0, \sigma^2 = \frac{2}{D^{(l-1)}}\right)$$

Versió uniforme

$$\theta_{ij}^{(l)} \sim U\left(-\sqrt{\frac{6}{D^{(l-1)}}}, \sqrt{\frac{6}{D^{(l-1)}}}\right)$$

Resum pràctic

Activació	Inicialització	Variància dels pesos (σ^2)
Lineal / Tanh	Xavier (Glorot)	$\frac{2}{D^{(l-1)} + D^{(l)}}$
Sigmoide	Xavier (Glorot)	$\frac{2}{D^{(l-1)} + D^{(l)}}$
ReLU	He (Kaiming)	$\frac{2}{D^{(l-1)}}$

Missatge clau

La inicialització no canvia l'arquitectura de la xarxa, però pot canviar molt la facilitat amb què aquesta arquitectura aprèn.

En aquesta pràctica utilitzarem sobretot activacions **ReLU**, de manera que esperem que la inicialització **He** sigui especialment adequada.

Què vol dir començar bé?

Una bona inicialització busca que la variància del senyal es mantingui estable.

- Si la variància disminueix capa a capa: el senyal es contrau i pot desaparèixer.
- Si la variància augmenta capa a capa: el senyal es dilata i pot explotar.
- Si la variància es manté en una escala raonable: l'entrenament és més estable.

Idea final

Bona inicialització $\implies$ senyal estable $\implies$ gradients útils $\implies$ aprenentatge més ràpid.

Imports i llavors

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam

np.random.seed(41)
tf.random.set_seed(41)
```

Pas 1: Inicialització manual amb NumPy

Comencem implementant les inicialitzacions manualment, sense utilitzar encara Keras.

Tasca: completa la funció següent.

```
def initialize_weights(fan_in, fan_out, method="he", distribution="normal"):  
  
    if method == "xavier":  
        variance = .....  
    elif method == "he":  
        variance = .....  
    elif method == "small":  
        variance = 0.01**2  
    elif method == "large":  
        variance = 2.0**2  
    else:  
        raise ValueError("method ha de ser: xavier, he, small o large")  
  
    if distribution == "normal":  
        std = .....  
        W = np.random.randn(fan_in, fan_out) * std  
  
    elif distribution == "uniform":  
        limit = .....  
        W = np.random.uniform(-limit, limit, size=(fan_in, fan_out))  
  
    else:  
        raise ValueError("distribution ha de ser: normal o uniform")  
  
    return W
```

Pas 2: Propagació d'activacions

Ara veurem què passa amb el senyal quan travessa moltes capes inicialitzades amb diferents mètodes.

```
def relu(z):  
    return np.maximum(0, z)  
  
def simulate_activation_flow(input_dim=100, hidden_dim=100,  
                             n_layers=30, method="he",  
                             n_samples=1000):  
  
    X = np.random.randn(n_samples, input_dim)  
    activations_std = []  
    activations_mean = []  
  
    H = X  
  
    for layer in range(n_layers):  
        fan_in = H.shape[1]  
        fan_out = hidden_dim  
  
        # Inicialitzar pesos de la capa actual  
        W = .....  
        # Pre-activació (combinació lineal)  
        z = .....  
        # Activació ReLU  
        H = .....  
  
        activations_mean.append(H.mean())  
        activations_std.append(H.std())  
  
    return activations_mean, activations_std
```

Pas 3: Visualitzar l'escala de les activacions

Utilitzem aquesta funció per comparar com evoluciona la desviació típica de les activacions.

```
def plot_activation_flow(methods):  
  
    plt.figure(figsize=(10, 4))  
  
    for method in methods:  
        means, stds = simulate_activation_flow(method=method)  
        plt.plot(stds, marker="o", label=method)  
  
    plt.yscale("log")  
    plt.title("Desviació típica de les activacions per capa")  
    plt.xlabel("Capa")  
    plt.ylabel("Desviació típica (escala log)")  
    plt.grid(True, alpha=0.3)  
    plt.legend()  
    plt.show()  
  
plot_activation_flow(["small", "large", "xavier", "he"])
```

Nota: Fem servir escala logarítmica perquè la inicialització `large` pot fer créixer molt la desviació típica i comprimir visualment la resta de corbes.

Pas 3: Què esperem veure?

En aquest gràfic volem veure si l'escala de les activacions es manté estable quan passem per moltes capes.

- `small`: la desviació típica baixa ràpidament cap a zero. El senyal es va apagant.
- `large`: la desviació típica creix molt ràpidament. El senyal explota.
- `xavier`: pot ser raonable, però amb ReLU sovint tendeix a reduir una mica l'escala.
- `he`: hauria de mantenir millor l'escala de les activacions amb ReLU.

Interpretació

Una inicialització útil no és la que dona pesos més grans, sinó la que manté el senyal en una escala raonable durant el forward pass.

Pas 4: Visualitzar la distribució dels pesos

També podem mirar directament la distribució dels pesos inicialitzats.

```
def plot_weight_histograms(fan_in=784, fan_out=128):  
    methods = ["small", "large", "xavier", "he"]  
    plt.figure(figsize=(12, 8))  
  
    for i, method in enumerate(methods):  
        W = initialize_weights(fan_in, fan_out, method=method)  
  
        plt.subplot(2, 2, i + 1)  
        plt.hist(W.flatten(), bins=50, alpha=0.8)  
        plt.title(f"{method}: std={W.std():.4f}")  
        plt.xlabel("Valor del pes")  
        plt.ylabel("Freq ència")  
        plt.grid(True, alpha=0.3)  
  
    plt.tight_layout()  
    plt.show()  
  
plot_weight_histograms(fan_in=30, fan_out=30)
```

Pas 4: Què esperem veure?

Els histogrames mostren l'escala inicial dels pesos.

- **small**: histograma molt concentrat al voltant de 0.
- **large**: histograma molt ample, amb pesos inicials molt grans.
- **xavier**: histograma centrat a 0 amb una amplada moderada.
- **he**: histograma també centrat a 0, normalment una mica més ample que Xavier quan $D^{(l-1)}$ i $D^{(l)}$ són semblants.

Interpretació

Tots els mètodes generen pesos aleatoris centrats a 0. La diferència important és la **variància**.

Pas 5: Dataset two_moons

Ara aplicarem les inicialitzacions a un problema ja treballat: classificació binària amb `two_moons`.

```
X, y = make_moons(n_samples=1000, noise=0.25, random_state=41)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=41, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_scaled = scaler.transform(X)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

Recordatori: escalem les dades perquè les variables d'entrada tinguin una escala comparable.

Pas 6: Inicialitzadors personalitzats en Keras

Ara definirem una funció que retorni l'inicialitzador de Keras corresponent.

```
def get_initializer(method):  
    if method == "xavier":  
        return tf.keras.initializers.GlorotNormal(seed=41)  
  
    elif method == "he":  
        return tf.keras.initializers.HeNormal(seed=41)  
  
    elif method == "small":  
        return tf.keras.initializers.RandomNormal(  
            mean=0.0, stddev=0.01, seed=41  
        )  
  
    elif method == "large":  
        return tf.keras.initializers.RandomNormal(  
            mean=0.0, stddev=2.0, seed=41  
        )  
  
    else:  
        raise ValueError("method ha de ser: xavier, he, small o large")
```

Pas 7: Construir una xarxa més profunda

Per veure millor les diferències entre inicialitzacions, utilitzarem una xarxa una mica més profunda que en pràctiques anteriors.

Arquitectura:

2 – 64 – 64 – 64 – 64 – 64 – 1

Com més capes tenim, més important és mantenir estable la variància del senyal.

Pas 7: Construir el model

Tasca: completa el model utilitzant l'inicialitzador indicat.

```
def build_moons_model(method):  
    initializer = get_initializer(method)  
  
    model = Sequential([  
        .....  
        .....  
        .....  
        .....  
        .....  
        .....  
    ])  
  
    model.compile(  
        optimizer=Adam(learning_rate=0.005),  
        loss="binary_crossentropy",  
        metrics=["accuracy"]  
    )  
  
    return model
```

Com que les capes ocultes utilitzen **ReLU**, esperem que he sigui una opció natural.

Pas 8: Entrenar diferents inicialitzacions

Entrenarem el mateix model diverses vegades, canviant només la inicialització.

```
def train_moons_with_initialization(method, epochs=100):  
    tf.random.set_seed(41)  
    model = build_moons_model(method)  
  
    history = model.fit(  
        X_train, y_train,  
        validation_split=0.2,  
        epochs=epochs,  
        batch_size=32,  
        verbose=0  
    )  
  
    loss_test, acc_test = model.evaluate(X_test, y_test, verbose=0)  
  
    return model, history, loss_test, acc_test  
  
methods = ["small", "large", "xavier", "he"]  
results = {}  
  
for method in methods:  
    model, history, loss_test, acc_test = train_moons_with_initialization(method)  
    results[method] = {  
        "model": model,  
        "history": history,  
        "loss_test": loss_test,  
        "acc_test": acc_test  
    }  
    print(method, "Test accuracy:", acc_test, "Test loss:", loss_test)
```


Pas 9: Visualitzar les corbes d'entrenament (1/3)

Definim una funció per visualitzar les corbes de **loss** i **accuracy**.

```
def plot_histories(results):  
  
    plt.figure(figsize=(12, 8))  
  
    # 1) Validation loss amb totes les inicialitzacions  
    plt.subplot(2, 2, 1)  
    for method, res in results.items():  
        plt.plot(res["history"].history["val_loss"], label=method)  
    plt.title("Validation loss")  
    plt.xlabel("Epoch")  
    plt.ylabel("Loss")  
    plt.grid(True, alpha=0.3)  
    plt.legend()  
  
    # 2) Validation loss sense large  
    plt.subplot(2, 2, 2)  
    for method, res in results.items():  
        if method != "large":  
            plt.plot(res["history"].history["val_loss"], label=method)  
    plt.title("Validation loss (sense large)")  
    plt.xlabel("Epoch")  
    plt.ylabel("Loss")  
    plt.grid(True, alpha=0.3)  
    plt.legend()
```

Pas 9: Visualitzar les corbes d'entrenament (2/3)

```
# 3) Validation accuracy amb totes
plt.subplot(2, 2, 3)
for method, res in results.items():
    plt.plot(res["history"].history["val_accuracy"], label=method)
plt.title("Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True, alpha=0.3)
plt.legend()

# 4) Validation accuracy sense large
plt.subplot(2, 2, 4)
for method, res in results.items():
    if method != "large":
        plt.plot(res["history"].history["val_accuracy"], label=method)
plt.title("Validation accuracy (sense large)")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()

plot_histories(results)
```

Pas 9: Què esperem veure?

Aquest gràfic mostra si la inicialització ajuda o dificulta l'entrenament.

- `large`: pot generar valors de **loss molt grans** i un entrenament inestable.
- `small`: pot aprendre lentament, perquè els pesos inicials són molt petits.
- `xavier` i `he`: donen un entrenament més estable.
- Amb ReLU, `he` acostuma a comportar-se bé, però pot ser similar a `xavier` en aquest problema.

Important

Separem el cas `large` perquè pot dominar l'escala del gràfic i dificultar la comparació amb la resta.

Idea clau

No només importa l'accuracy final, sinó si la loss baixa de manera estable i si el model evita comportaments extrems com `small` o `large`.

Pas 10: Visualitzar la frontera de decisió

Utilitzarem la mateixa idea que en pràctiques anteriors: representar la predicció del model sobre una malla de punts.

```
def plot_decision_boundary(X, y, model, title):  
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5  
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5  
  
    xx, yy = np.meshgrid(  
        np.linspace(x_min, x_max, 300),  
        np.linspace(y_min, y_max, 300)  
    )  
  
    grid = np.c_[xx.ravel(), yy.ravel()]  
    probs = model.predict(grid, verbose=0).reshape(xx.shape)  
  
    plt.figure(figsize=(7, 5))  
    plt.contourf(xx, yy, probs, alpha=0.3, cmap=plt.cm.Spectral)  
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k", cmap=plt.cm.Spectral)  
    plt.title(title)  
    plt.show()  
  
for method in methods:  
    plot_decision_boundary(  
        X_scaled,  
        y,  
        results[method]["model"],  
        title=f"Inicialització: {method}"  
    )
```

Pas 10: Què esperem veure?

Les fronteres de decisió ens permeten veure visualment si el model ha après la forma no lineal de `two_moons`.

- Amb una bona inicialització, la frontera hauria de separar aproximadament les dues llunes.
- Amb `small`, la frontera pot quedar massa simple o aprendre més lentament.
- Amb `large`, la frontera pot ser més irregular o l'entrenament pot ser menys estable.
- Xavier i He haurien de donar fronteres més netes i coherents.

Experimentació

Un cop tingueu la pràctica funcionant, proveu modificacions controlades.

Experiments recomanats

- Canvieu el nombre de capes ocultes i observeu si la inicialització es torna més important.
- Compareu `xavier` i `he` amb activacions `relu`.
- Proveu `tanh` i observeu si Xavier millora respecte a He.
- Canvieu el `learning rate` i mireu si una mala inicialització fa l'entrenament més sensible.

Important: canvieu una sola cosa cada vegada. Si modifiqueu inicialització, arquitectura i `learning rate` alhora, serà difícil interpretar el resultat.

Després d'executar el vostre codi, responeu:

- Per què inicialitzem els pesos com a variables aleatòries?
- Per què normalment fem servir distribucions centrades a zero?
- Per què una inicialització massa petita pot frenar l'aprenentatge?
- Per què una inicialització massa gran pot fer l'entrenament inestable?
- Quina diferència hi ha entre Xavier i He?
- Per què He és especialment adequada per a capes amb ReLU?
- Què passa si augmenteu molt el nombre de capes ocultes?

Conclusió

Aquesta pràctica mostra que entrenar una xarxa neuronal no depèn només de l'arquitectura i de l'optimitzador.

Missatge clau

La inicialització determina el punt de partida de l'entrenament i l'escala inicial del senyal. Una bona inicialització ajuda a mantenir estable el forward pass i el backward pass.

En general:

- **Xavier/Glorot** és una bona opció per a activacions simètriques com $\tanh$ i lineals.
- **He/Kaiming** és una bona opció per a xarxes amb activació ReLU.
- Inicialitzacions massa petites o massa grans poden dificultar molt l'entrenament.